

Computer Architecture and Organization Lab

Rollno: 24MAI0065

Name: NITHISH BS

I. Write an OpenMP Program for the following:

a) To print “Hello World” in multiple threads.

Program Code:

```
#include
<stdio.h>#include
<omp.h>int
main()
{
omp_set_num_threads(4);
#pragma omp parallel
{
printf("Hello World from thread %d\n",omp_get_thread_num());
}
return 0;
}
```

Code screenshot:

```
nithish@nithish-VirtualBox: ~  
#include <stdio.h>  
#include <omp.h>  
int main()  
{  
    omp_set_num_threads(4);  
    #pragma omp parallel  
    {  
        printf("Hello World from thread %d\n",omp_get_thread_num());  
    }  
    return 0;  
}
```

Output screenshot:

```
nithish@nithish-VirtualBox: ~  
nithish@nithish-VirtualBox:~$ vi nithish.c  
nithish@nithish-VirtualBox:~$ vi nithish.c  
nithish@nithish-VirtualBox:~$ gcc-openmp nithish.c  
gcc-openmp: command not found  
nithish@nithish-VirtualBox:~$ gcc-fopenmp nithish.c  
gcc-fopenmp: command not found  
nithish@nithish-VirtualBox:~$ gcc -fopenmp -o nithish.c  
gcc: fatal error: no input files  
compilation terminated.  
nithish@nithish-VirtualBox:~$ gcc -fopenmp -o nithish nithish.c  
nithish@nithish-VirtualBox:~$ ./a.out  
bash: ./a.out: No such file or directory  
nithish@nithish-VirtualBox:~$ ./nithish  
Hello World from thread 3  
Hello World from thread 0  
Hello World from thread 1  
Hello World from thread 2  
nithish@nithish-VirtualBox:~$ vi nithish.c  
nithish@nithish-VirtualBox:~$ gcc -fopenmp -o nithish nithish.c  
nithish@nithish-VirtualBox:~$ vi nithish.c  
nithish@nithish-VirtualBox:~$ gcc -fopenmp -o nithish nithish.c  
nithish@nithish-VirtualBox:~$ ./nithish  
Hello World from thread 3  
Hello World from thread 2  
Hello World from thread 0  
Hello World from thread 1  
nithish@nithish-VirtualBox:~$
```

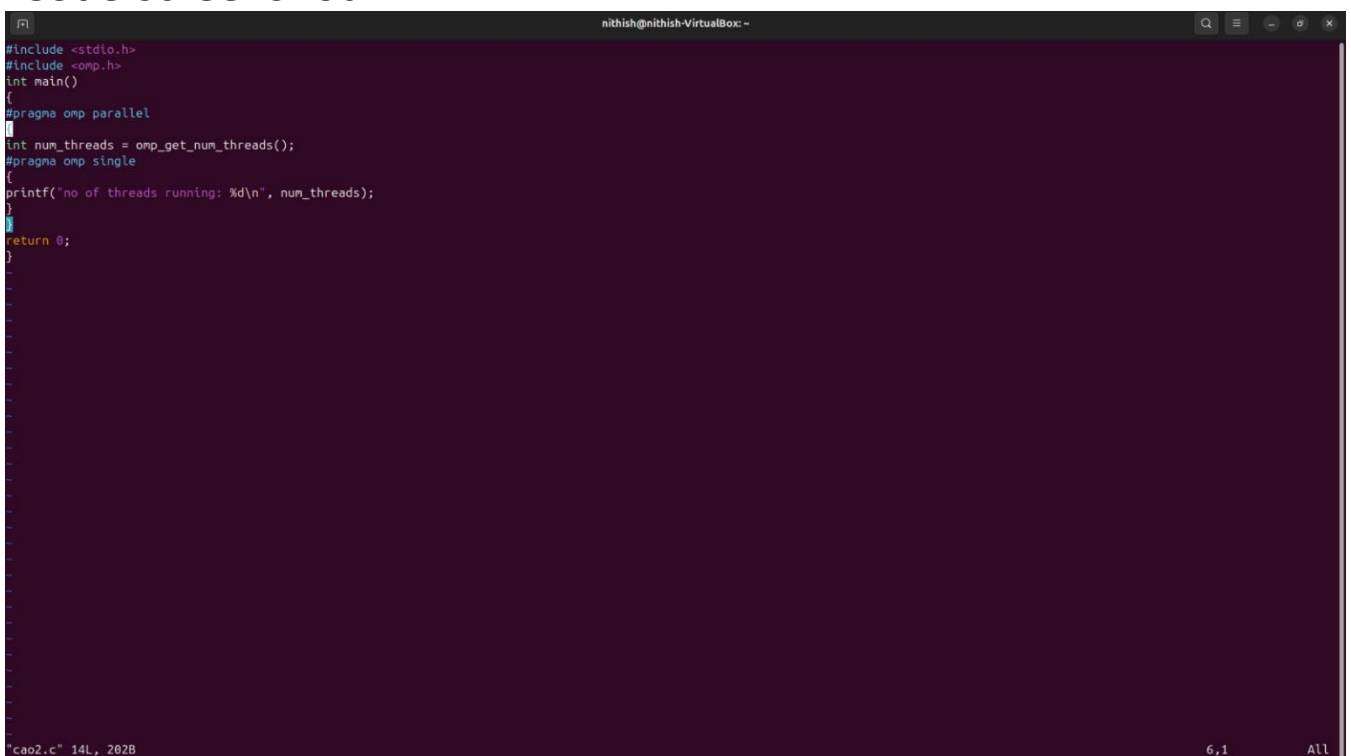
b) To find number of threads running currently

Code:

```
#include <stdio.h>  
#include <omp.h>  
int main()  
{
```

```
#pragma omp parallel {  
  
    int num_threads = omp_get_num_threads();  
  
    #pragma omp single  
  
    {  
        printf("no of threads running: %d\n", num_threads);  
    }  
  
}  
  
return 0;  
  
}
```

Code screenshot:

A screenshot of a code editor window titled "nithish@nithish-VirtualBox: -". The editor shows the C program code from the previous block. The code is written in a dark-themed editor with a dark blue background. The code is as follows:

```
#include <stdio.h>  
#include <omp.h>  
int main()  
{  
    #pragma omp parallel  
    {  
        int num_threads = omp_get_num_threads();  
        #pragma omp single  
        {  
            printf("no of threads running: %d\n", num_threads);  
        }  
    }  
    return 0;  
}
```

The status bar at the bottom left shows "cao2.c" 14L, 262B. The status bar at the bottom right shows "6,1" and "All".

Output screenshot:

```
nithish@nithish-VirtualBox: ~$ gcc -fopenmp -o cao2 cao2.c
nithish@nithish-VirtualBox: ~$ ./cao2
no of threads running: 2
nithish@nithish-VirtualBox: ~$
```

c) To find maximum number of threads.

Program Code:

```
#include <stdio.h>

#include <omp.h>

int main()

{

#pragma omp parallel

{

int max_threads = omp_get_max_threads();

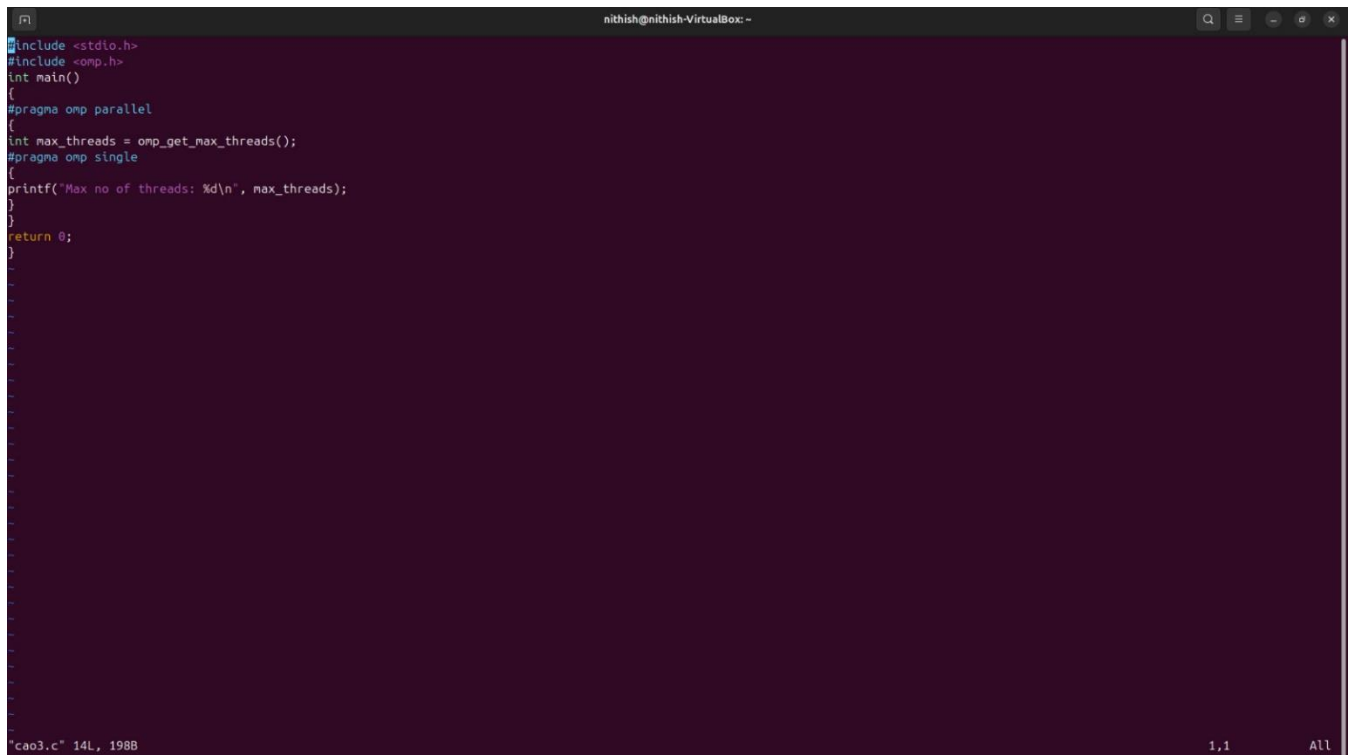
#pragma omp single

{

printf("Max no of threads: %d\n", max_threads);
```

```
}  
}  
  
return 0;  
  
}
```

Code screenshot:



The screenshot shows a code editor window titled "nithish@nithish-VirtualBox: -". The code is as follows:

```
#include <stdio.h>  
#include <omp.h>  
int main()  
{  
    #pragma omp parallel  
    {  
        int max_threads = omp_get_max_threads();  
        #pragma omp single  
        {  
            printf("Max no of threads: %d\n", max_threads);  
        }  
    }  
    return 0;  
}
```

The status bar at the bottom indicates the file is "cao3.c", 14 lines long, 1988 bytes, at line 1, column 1, with all files saved.

Output screenshot:

```
nithish@nithish-VirtualBox: ~$ vi cao3.c
nithish@nithish-VirtualBox: ~$ gcc -fopenmp -o cao3 cao3.c
nithish@nithish-VirtualBox: ~$ ./cao3
Max no of threads: 2
nithish@nithish-VirtualBox: ~$
```

d) To find the thread id.

Program Code:

```
#include<stdio.h>

#include <omp.h>

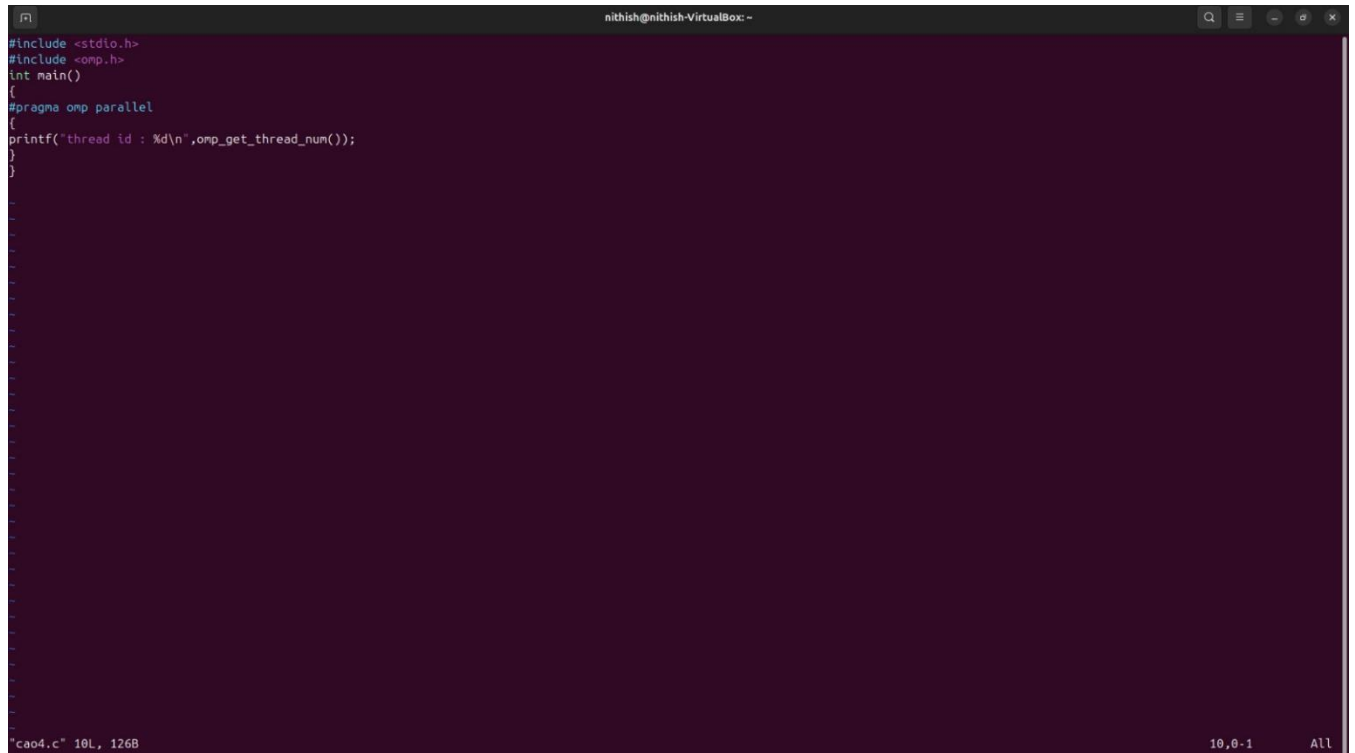
int main()

{
#pragma omp parallel
{
printf("thread id : %d\n",omp_get_thread_num());
```

```
}
```

```
}
```

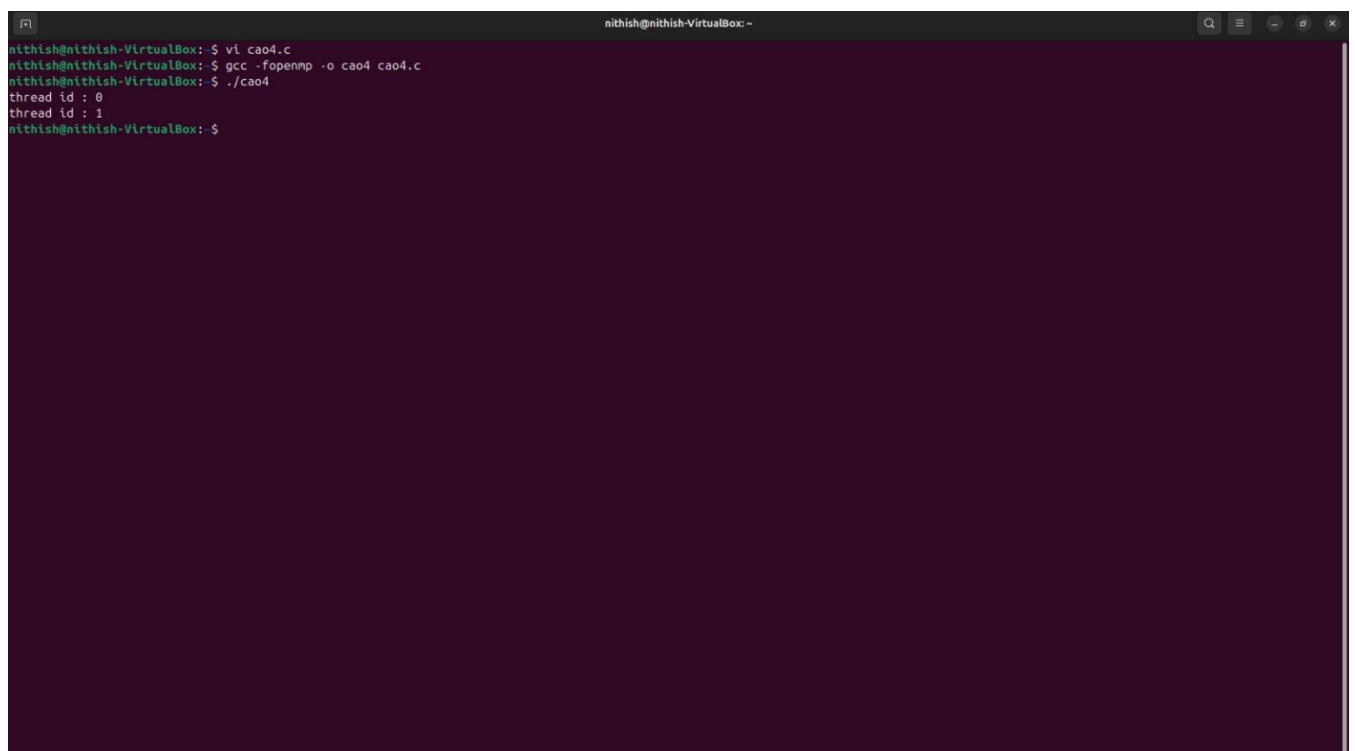
Code screenshot:



```
nithish@nithish-VirtualBox: -
#include <stdio.h>
#include <omp.h>
int main()
{
#pragma omp parallel
{
printf("thread id : %d\n",omp_get_thread_num());
}
}
```

"cao4.c" 10L, 126B 10,0-1 All

Output screenshot:



```
nithish@nithish-VirtualBox: -
nithish@nithish-VirtualBox: $ vi cao4.c
nithish@nithish-VirtualBox: $ gcc -fopenmp -o cao4 cao4.c
nithish@nithish-VirtualBox: $ ./cao4
thread id : 0
thread id : 1
nithish@nithish-VirtualBox: $
```

e) To find the number of processor core in system

Program Code:

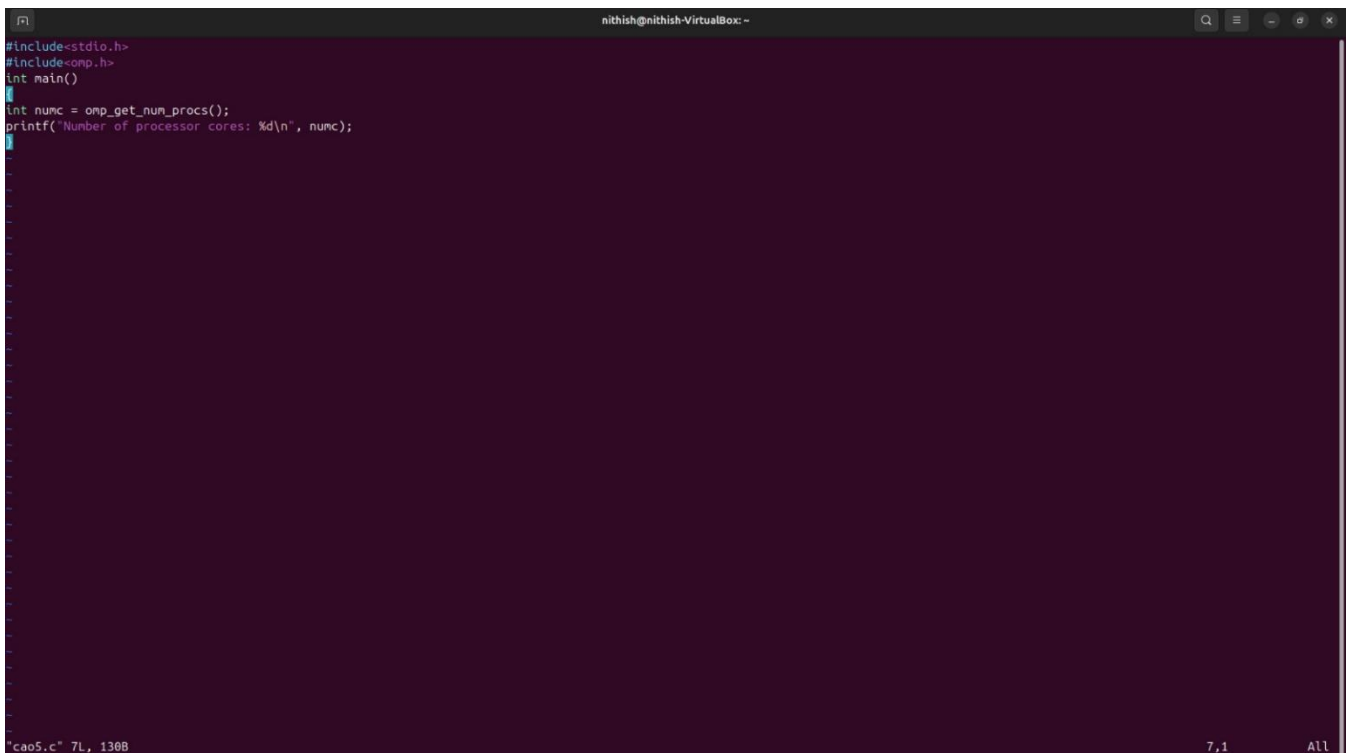
```
#include<stdio.h>

#include<omp.h>

int main()
{
    int numc = omp_get_num_procs();

    printf("Number of processor cores: %d\n",numc);
}
```

Code screenshot:

A screenshot of a code editor window titled "nithish@nithish-VirtualBox: -". The editor shows the same C code as in the previous block. The code is:

```
#include<stdio.h>
#include<omp.h>
int main()
{
    int numc = omp_get_num_procs();
    printf("Number of processor cores: %d\n", numc);
}
```

 The editor has a dark theme with a purple background. The status bar at the bottom shows "caos.c" 7L, 1308, 7,1, and All.

Output screenshot:


```
nithish@nithish-VirtualBox: ~$ vi cao5.c
nithish@nithish-VirtualBox: ~$ gcc -fopenmp -o cao5 cao5.c
nithish@nithish-VirtualBox: ~$ ./cao5
Number of processor cores: 2
nithish@nithish-VirtualBox: ~$
```

f) To set number of thread to be executed

Program Code:

```
#include<stdio.h>
```

```
#include <omp.h>
```

```
int main()
```

```
{
```

```
int num_threads = 4; // Number of threads to be set
```

```
// Set the number of threads
```

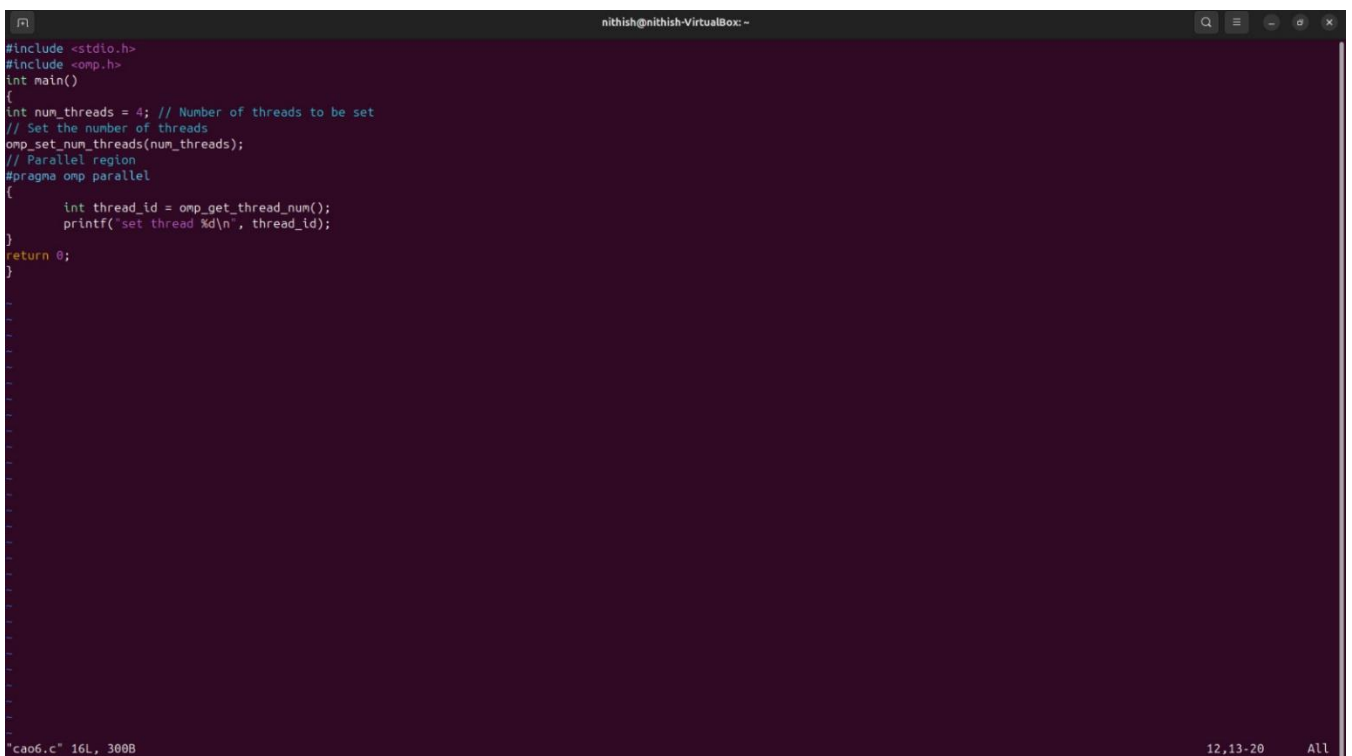
```
omp_set_num_threads(num_threads);
```

```
// Parallel region
```

```
#pragma omp parallel
```

```
{  
  
int thread_id = omp_get_thread_num();  
  
    printf("set thread %d\n", thread_id);  
  
}  
  
return 0;  
  
}
```

Code screenshot:

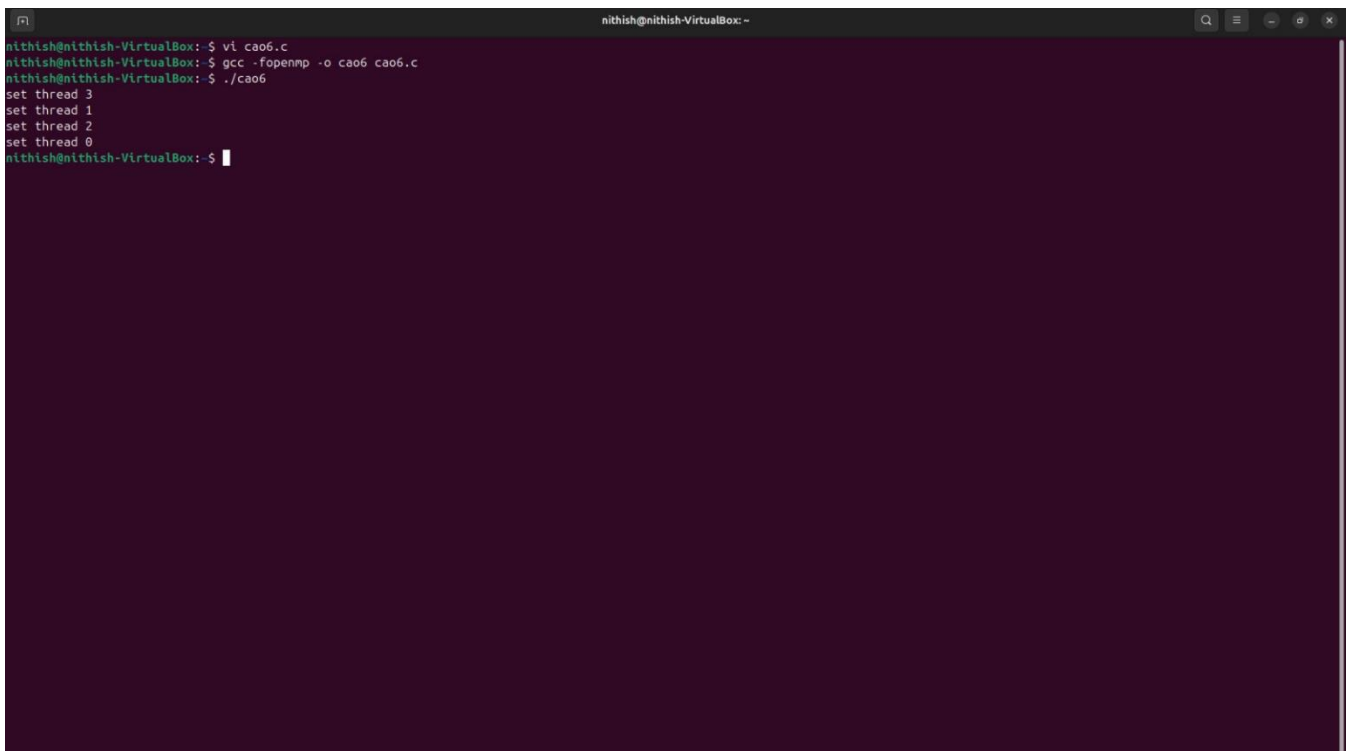


The screenshot shows a terminal window titled "nithish@nithish-VirtualBox: -". The code displayed is a C program that uses OpenMP to set the number of threads to 4 and prints the thread ID for each thread in a parallel region. The code is as follows:

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int num_threads = 4; // Number of threads to be set
    // Set the number of threads
    omp_set_num_threads(num_threads);
    // Parallel region
    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        printf("set thread %d\n", thread_id);
    }
    return 0;
}
```

The terminal output shows the program running successfully, with the thread IDs 0, 1, 2, and 3 printed. The status bar at the bottom indicates the file "cao6.c" is 16L, 300B, and the cursor is at line 12, column 13-20.

Output screenshot:



```
nithish@nithish-VirtualBox: ~$ vi cao6.c
nithish@nithish-VirtualBox: ~$ gcc -fopenmp -o cao6 cao6.c
nithish@nithish-VirtualBox: ~$ ./cao6
set thread 3
set thread 1
set thread 2
set thread 0
nithish@nithish-VirtualBox: ~$
```

g) To test is_parallel function.

Program Code:

```
#include<stdio.h>
```

```
#include <omp.h>
```

```
void test_is_parallel()
```

```
{
```

```
if(omp_in_parallel())
```

```
{
```

```
    printf("We are in a parallel region.\n");}
```

```
else {
```

```
    printf("We are not in a parallel region.\n");}
```

```
}

int main() {

printf("Outside parallel region:\n");

test_is_parallel();

// Parallel region

#pragma omp

parallel{

// Ensure only one thread prints this

#pragma omp single{

printf("Inside parallel region:\n");

test_is_parallel();}

}

return 0;
```

Code screenshot:

```
nithish@nithish-VirtualBox: ~  
#include <stdio.h>  
#include <omp.h>  
void test_is_parallel() {  
    if (omp_in_parallel()) {  
        printf("We are in a parallel region.\n");  
    }else {  
        printf("We are not in a parallel region.\n");  
    }  
}  
  
int main() {  
    printf("Outside parallel region:\n");  
    test_is_parallel();  
    #pragma omp parallel  
    {  
        #pragma omp single  
        {  
            printf("Inside parallel region:\n");  
            test_is_parallel();  
        }  
    }  
    return 0;  
}
```

"cao7.c" 22L, 410B 12,20-27 All

Output screenshot:

```
nithish@nithish-VirtualBox: ~  
nithish@nithish-VirtualBox: $ vi cao7.c  
nithish@nithish-VirtualBox: $ gcc -fopenmp -o cao7 cao7.c  
nithish@nithish-VirtualBox: $ ./cao7  
Outside parallel region:  
We are not in a parallel region.  
Inside parallel region:  
We are in a parallel region.  
nithish@nithish-VirtualBox: $
```

h) To parallelize a simple for loop.

Program Code:

```
#include<stdio.h>

#include<omp.h>

int main()

{
int i;

#pragma omp parallel

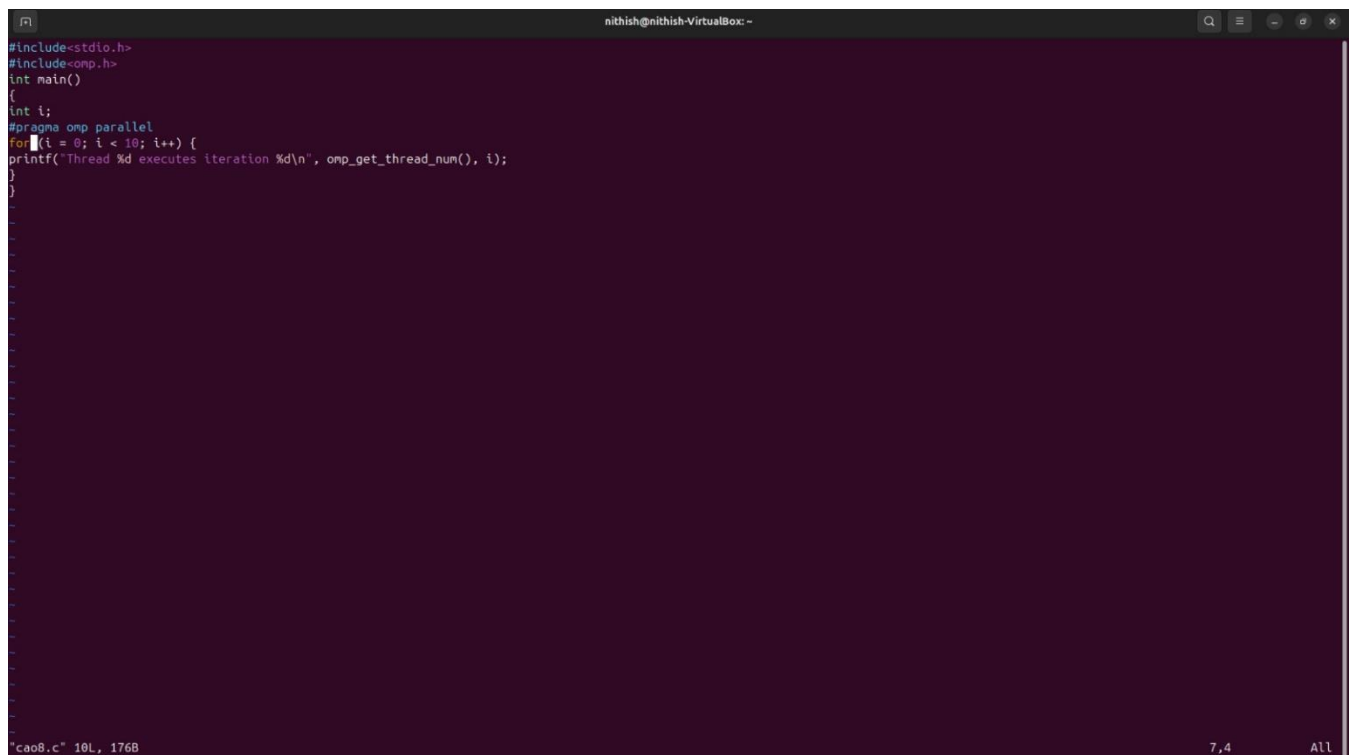
for (i = 0; i < 10; i++) {

printf("Thread %d executes iteration %d\n",
omp_get_thread_num(), i);

}

}
```

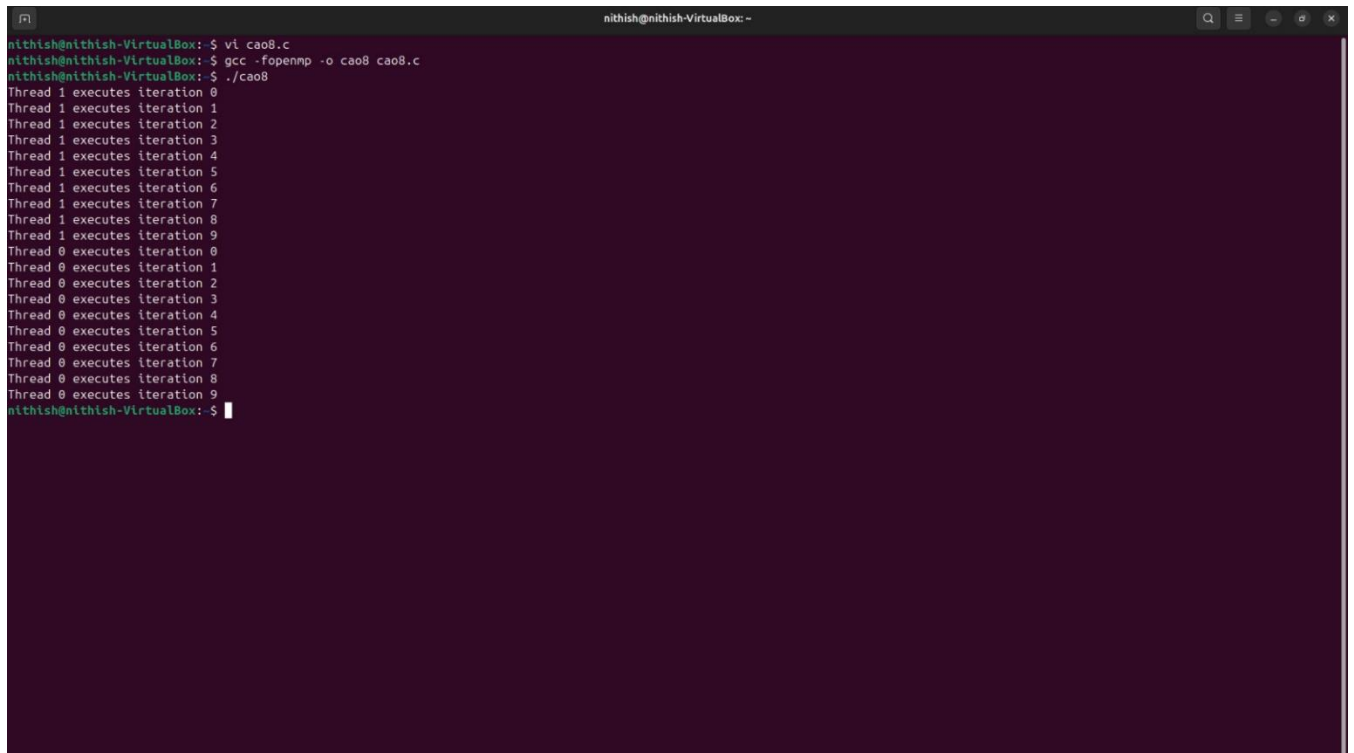
Code screenshot:

A screenshot of a terminal window with a dark purple background. The window title is "nithish@nithish-VirtualBox: ~". The code displayed is the same as in the previous block, with syntax highlighting: preprocessor directives in green, keywords in blue, and identifiers in white. The code is:

```
#include<stdio.h>
#include<omp.h>
int main()
{
int i;
#pragma omp parallel
for (i = 0; i < 10; i++) {
printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
}
}
```

 At the bottom left, it says "cao8.c" 10L, 176B. At the bottom right, it shows "7,4" and "All".

Output screenshot:



```
nithish@nithish-VirtualBox: $ vi cao8.c
nithish@nithish-VirtualBox: $ gcc -fopenmp -o cao8 cao8.c
nithish@nithish-VirtualBox: $ ./cao8
Thread 1 executes iteration 0
Thread 1 executes iteration 1
Thread 1 executes iteration 2
Thread 1 executes iteration 3
Thread 1 executes iteration 4
Thread 1 executes iteration 5
Thread 1 executes iteration 6
Thread 1 executes iteration 7
Thread 1 executes iteration 8
Thread 1 executes iteration 9
Thread 0 executes iteration 0
Thread 0 executes iteration 1
Thread 0 executes iteration 2
Thread 0 executes iteration 3
Thread 0 executes iteration 4
Thread 0 executes iteration 5
Thread 0 executes iteration 6
Thread 0 executes iteration 7
Thread 0 executes iteration 8
Thread 0 executes iteration 9
nithish@nithish-VirtualBox: $
```